

算法设计与分析

Design & Analysis of Algorithms

第6章 分支限界法

第6章 分支限界法

6.1 分支限界法的基本思想

6.2 求解装载问题

6.3 求解0/1背包问题

6.4 最大团问题

6.5 求解旅行商问题

6.6 小结

回溯法回顾

- 回溯法是一种通用性解法，可以将回溯法看作是带优化的穷举法。
- 回溯法的基本思想是在一棵含有问题全部可能解的状态空间树上进行**深度优先搜索**。搜索过程中，每到达一个结点时，则判断*该结点为根的子树*是否含有问题的解，如果可以确定该子树中不含有问题的解，则**放弃对该子树的搜索，退回到上层父结点**，继续下一步深度优先搜索过程。
- 在回溯法中，并不是先构造出整棵状态空间树，再进行搜索，而是在搜索过程，逐步构造出状态空间树，即**边搜索，边构造**。

优化穷举 碰壁回头 动态构造

回溯法求解步骤

- (1)针对所给问题，定义问题的**解空间**；
- (2)确定易于搜索的**解空间结构（子集树和排列树）**；
- (3)以深度优先方式搜索解空间，并在搜索过程中用**剪枝函数**避免无效搜索。

常用剪枝函数

1. 用**约束函数**在扩展结点处剪去不满足约束的子树；
2. 用**限界函数**剪去得不到最优解的子树。

6.1 分支限界法的基本思想

分支限界法基本思想

- 分支限界法常以**广度优先**或以**最小耗费（最大效益）优先**的方式搜索问题的解空间树。
- 在分支限界法中，**每一个活结点只有一次机会成为扩展结点**。活结点一旦成为扩展结点，就一次性产生其所有儿子结点。在这些儿子结点中，导致不可行解或导致非最优解的儿子结点被舍弃，其余儿子结点被加入活结点表中。
- 此后，**从活结点表中取下一结点成为当前扩展结点**，并重复上述结点扩展过程。这个过程一直持续到找到所需的解或活结点表为空时为止。

6.1 分支限界法的基本思想

分支限界法与回溯法的不同

- (1) **求解目标**：回溯法的求解目标是找出解空间树中满足约束条件的所有解，而分支限界法的求解目标则是找出满足约束条件的一个解，或是在满足约束条件的解中找出在某种意义下的最优解。
- (2) **搜索方式的不同**：回溯法以**深度优先**的方式搜索解空间树，而分支限界法则以**广度优先**或以**最小耗费优先**的方式搜索解空间树。

6.1 分支限界法的基本思想

常见的两种分支限界法

(1) 队列式(FIFO)分支限界法

- 按照队列先进先出 (FIFO) 原则选取下一个节点为扩展节点;
- 数据结构: 队列

(2) 优先队列式分支限界法

- 按照优先队列中规定的优先级选取优先级最高的节点成为当前扩展节点。
- 数据结构: 堆
 - 最大优先队列: 使用最大堆, 体现最大效益优先
 - 最小优先队列: 使用最小堆, 体现最小费用优先

6.1分支限界法的基本思想

□ 队列式分支限界法程序框架

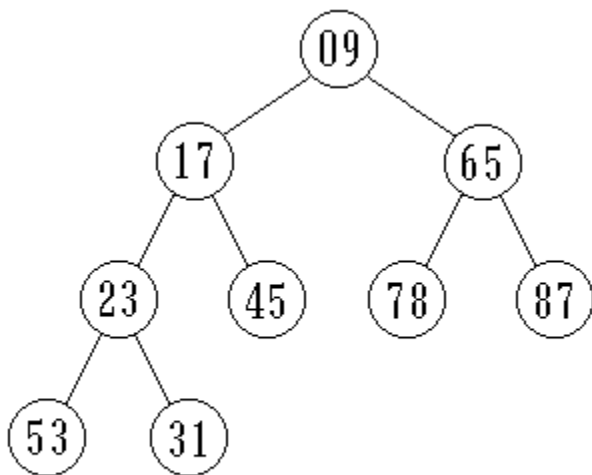
1. 设 T 为状态空间树的根结点；初始化队列 Q ；
2. 将 T 入队；
3. 循环，直到队列 Q 空（无解）：
4. 结点 e 出队；
5. 若 e 是回答结点，则
6. 输出解或求解路径；
7. 否则
8. 检查 e 的所有子结点 x ：
9. 若 x 满足约束条件，则
10. 将 x 入队；
11. 记录搜索路径；

知识点：堆

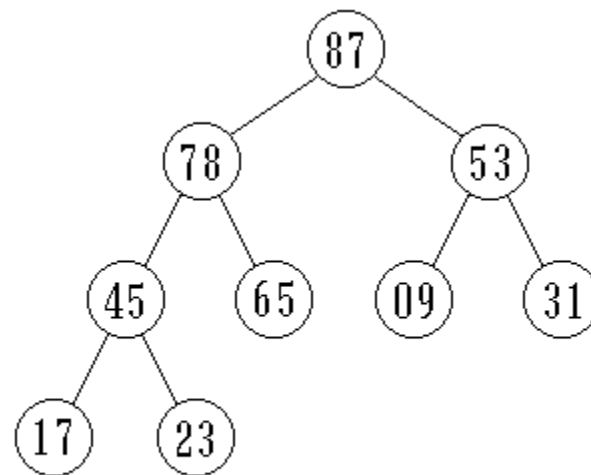
n 个元素的序列 $\{k_1, k_2, \dots, k_n\}$

当且仅当满足下述关系时，称之为堆

$$\begin{cases} k_i \leq k_{2i} \\ k_i \leq k_{2i+1} \end{cases} \quad \text{或} \quad \begin{cases} k_i \geq k_{2i} \\ k_i \geq k_{2i+1} \end{cases} \quad i = 1, 2, \dots, \left\lfloor \frac{n}{2} \right\rfloor$$



(a) 最小堆



(b) 最大堆

6.1分支限界法的基本思想

- 优先队列式分支限界法程序框架

1. 设 T 为状态空间树的根结点； $\sim C(x)$ 为耗费估计函数；初始化优先队列 Q ；
2. 计算 $\sim C(T)$ ，并将 T 入队；
3. 循环，直到队列 Q 空（无解）：
 4. 结点 e 出队；
 5. 若 e 是回答结点，则
 6. 输出解或求解路径，求解结束；
 7. 否则
 8. 检查 e 的所有子结点 x ：
 9. 若 x 满足约束条件，则
 10. 计算 $\sim C(x)$ ，并将 x 入队；
 11. 记录搜索路径；

6.2 装载问题

有一批 n 个集装箱要装上2艘载重量分别为 C_1 和 C_2 的轮船，其中集装箱 i 的重量为 W_i ，且 $\sum_{i=1}^n w_i \leq c_1 + c_2$

采用下面的策略可得到最优装载方案。

- (1) 将第一艘轮船尽可能装满；
- (2) 将剩余集装箱装上第二艘轮船；

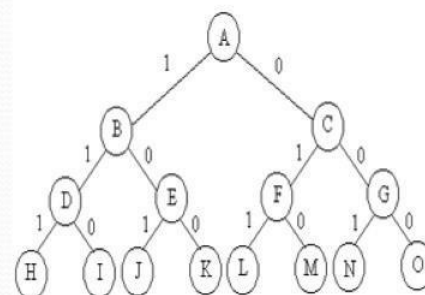
6.2 装载问题

1. 队列式分支限界法

- (1) 检测当前扩展结点的左儿子结点，可行入队。
- (2) 将其右儿子结点加入队列(右儿子结点一定可行)。
- (3) 舍弃当前扩展结点。
- (4) 每层结点之后都加一个尾部标记-1，将活结点分层。

只考虑求最大载重量，不考虑构造最优解

$C=10$, $W1=3$,
 $W2=4$, $W3=5$



-1	3	0							i=1
	3	0	-1						
			-1	7	3	4	0		i=2
				7	3	4	0	-1	
				<u>12</u> 7	8 3	<u>9</u> 4	5 0		i=3

bestw=9

装载问题队列式分支限界法算法

Ew=0, Q.Add(-1) ;i=1; //Ew为当前结点对应的在船的集装箱重量

while (true) {

 // 检查左儿子结点

 if (Ew + w[i] <= c) // x[i] = 1

 EnQueue(Q, Ew + w[i], bestw, i, n);

 // 右儿子结点总是可行的

 EnQueue(Q, Ew, bestw, i, n); // x[i] = 0

 Q.Delete(Ew); // 取下一扩展结点

 if (Ew == -1) { // 同层结点尾部

 if (Q.IsEmpty()) return bestw;

 Q.Add(-1); // 同层结点尾部标志

 Q.Delete(Ew); // 取下一扩展结点

 i++; // 进入下一层

}

```
template<class Type>
```

```
void EnQueue(Queue<Type> &Q, Type wt,  
Type &bestw, int i, int n)
```

```
{ if(i==n){
```

```
    if(wt>bestw)bestw=wt;
```

```
    } //叶子结点不入队列
```

```
    else Q.add(wt);
```

```
}
```

6.2 装载问题

2. 算法的改进（右子树加入剪枝条件）

- 上述算法中右子没有剪枝，效率较差。
- 策略：设bestw是当前最优解；ew是当前扩展结点所相应的重量；r是剩余集装箱的重量。则当 $ew + r \leq bestw$ 时，可将其右子树剪去。
- 为确保右子树成功剪枝，算法每一次进入左子树的时候更新bestw的值。不要等待 $i = n$ 时才去更新。

2. 算法的改进

// 检查左儿子结点

Type wt = Ew + w[i]; // 左儿子结点的重量

if (wt <= c) { // 可行结点

if (wt > bestw) bestw = wt;

if (i < n) Q.Add(wt); // 加入活结点队列

}

提前更新bestw

右儿子剪枝

// 检查右儿子结点

if (Ew + r > bestw && i < n)

Q.Add(Ew); // 可能含最优解

6.2 装载问题

3. 构造最优解

为在算法结束后能方便地构造出与最优值相应的最优解，可在每个结点处设置指向其父结点的指针，并设置左、右儿子标志。

```
class QNode
{ QNode *parent; // 指向父结点的指针
  bool LChild;    // 左儿子标志
  Type weight;    // 结点所相应的载重量
}
```


6.2 装载问题

找到最优值后，可以根据parent回溯到根节点，找到最优解

// 构造当前最优解

```
for (int j = n ; j >= 1; j--)
```

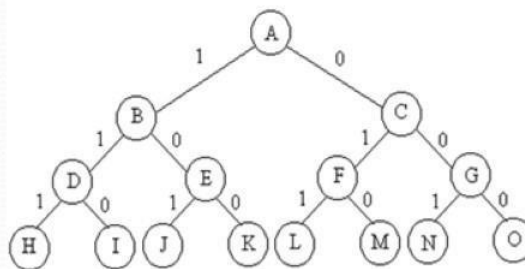
```
{
```

```
    bestx[j] = bestE->LChild;
```

```
    bestE = bestE->parent;
```

```
}
```

C=10, W1=3, W2=4, W5=5



解空间树中L结点构造出最优值，根据构造最优解的算法，最优解bestx[1..3]={0,1,1}

6.2 装载问题

4. 优先队列式分支限界法

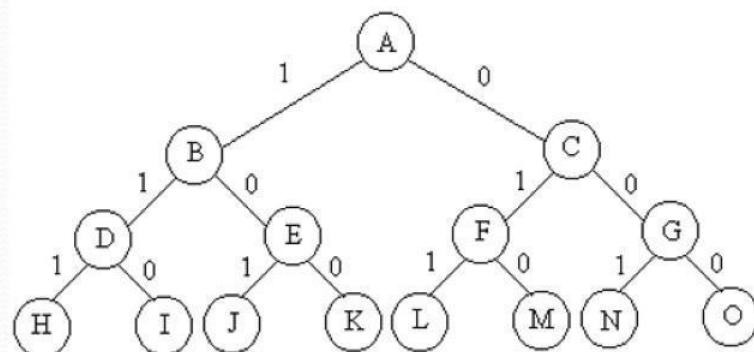
- 用最大优先队列存储活结点表（大顶堆）。
- 活结点x的优先级：**根到结点x的路径相应的载重量+剩余集装箱重量之和。**
- 优先队列中优先级最大的活结点成为下一个扩展结点。
- **以结点x为根的子树中所有结点相应的路径的载重量不会超过x的优先级。**
叶结点所相应的载重量与其优先级相同。
- 因此：**一旦优先队列中有一个叶结点成为当前扩展结点，则可以断言该叶结点所相应的解即为最优解。此时可终止算法。**

（注意：算法中叶子结点要进队列）

模拟优先队列（堆） 6.2 装载问题

3	0				载重				
1	0				解				
12	9				优先级				
2	2				下一层				

$n=3, C=10, W_1=3, W_2=4, W_3=5$

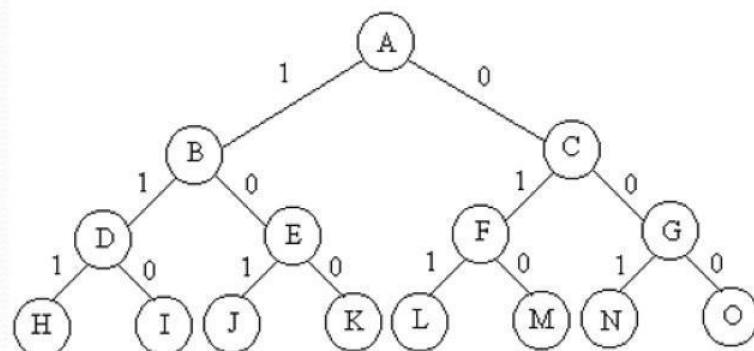


模拟优先队列（堆） 6.2 装载问题

3	0				载重				
1	0				解				
12	9				优先级				
2	2				下一层				

7	0	3			载重				
11	0	10			解				
12	9	8			优先级				
3	2	3			下一层				

$n=3, C=10, W_1=3, W_2=4, W_3=5$



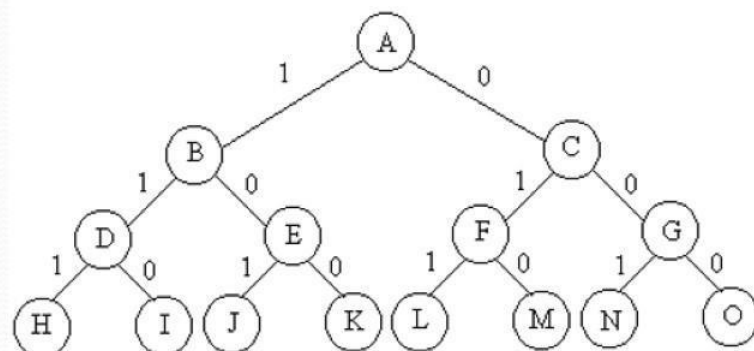
模拟优先队列（堆） 6.2 装载问题

3	0				载重				
1	0				解				
12	9				优先级				
2	2				下一层				

7	0	3			载重				
11	0	10			解				
12	9	8			优先级				
3	2	3			下一层				

0	3	7			载重				
0	10	110			解				
9	8	7			优先级				
2	3	4			下一层				

$n=3, C=10, W1=3, W2=4, W3=5$



模拟优先队列（堆） 6.2 装载问题

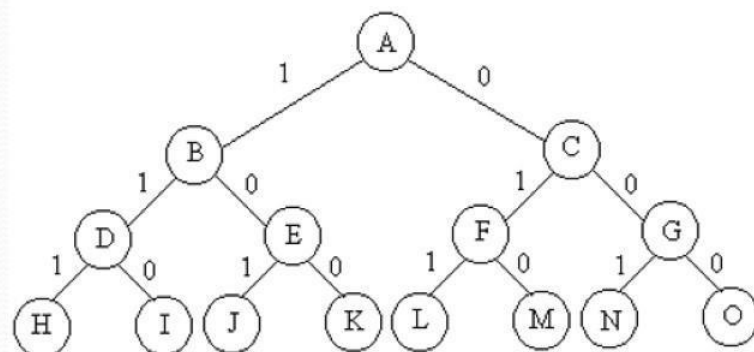
3	0			载重				
1	0			解				
12	9			优先级				
2	2			下一层				

7	0	3		载重				
11	0	10		解				
12	9	8		优先级				
3	2	3		下一层				

0	3	7		载重				
0	10	110		解				
9	8	7		优先级				
2	3	4		下一层				

4	3	7		载重				
01	10	110		解				
9	8	7		优先级				
3	3	4		下一层				

$n=3, C=10, W1=3, W2=4, W3=5$



模拟优先队列（堆） 6.2 装载问题

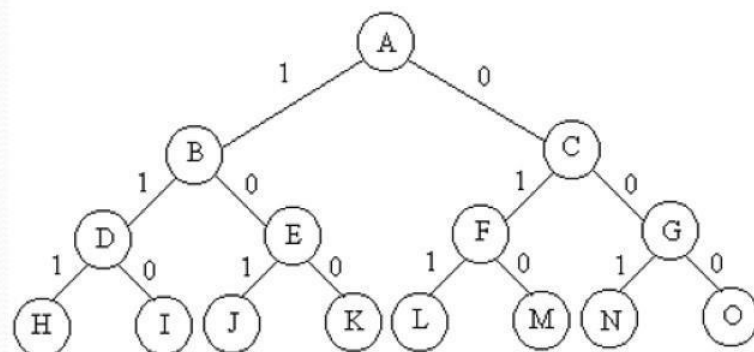
3	0			载重				
1	0			解				
12	9			优先级				
2	2			下一层				

7	0	3		载重				
11	0	10		解				
12	9	8		优先级				
3	2	3		下一层				

0	3	7		载重				
0	10	110		解				
9	8	7		优先级				
2	3	4		下一层				

4	3	7		载重				
01	10	110		解				
9	8	7		优先级				
3	3	4		下一层				

$n=3, C=10, W1=3, W2=4, W3=5$



9	3	7	载重	
011	10	110	解	
9	8	7	优先级	
4	3	4	下一层	

实验6-1: 装载问题(分支限界法)

- 问题描述: 有一批共 n 个集装箱要装上两艘载重量分别为 c_1 和 c_2 的轮船, 其中集装箱 i 的重量为 w_i , 且 $w_1 + w_2 + \dots + w_n \leq c_1 + c_2$ 。装载问题要求确定是否有一个合理的装载方案可将这些集装箱装上这两艘轮船。如果有, 找出一种装载方案。

- 示例 1:

当 $n=3$, $c_1=c_2=50$, $w=\{10, 40, 40\}$ 时, 则可以将集装箱1和2装到第一艘轮船上, 而将集装箱3装到第二艘轮船上。如果 $w=\{20, 40, 40\}$, 则无法将这3个集装箱都装上轮船。

- 要求:

1 填空完成程序;

2 输出装载结果。

*3 报告中指出是队列式还是优先队列式。

优先队列式分支限界法求解装载问题时，活结点表的组织形式是（ ）。

- ☐ A 最小堆
- ☒ B 最大堆
- ☐ C 栈
- ☐ D 数组

提交

6.3 0-1背包问题

- 算法的思想

- 先进行预处理：将各物品依其单位重量价值从大到小排列。
- 优先队列的优先级：已装物品价值+后面物品装满剩余容量的价值
- 算法：先检查当前扩展结点的左儿子结点。如果该左儿子结点是可行结点，则将它加入活结点优先队列中，如优于当前最优值，则更新当前最优值。当前扩展结点的右儿子结点一定是可行结点，仅当右儿子结点满足上界约束时（优先级大于当前最优值）才将它加入活结点优先队列。从优先队列中取下一个活结点成为扩展结点，继续扩展。
- 当叶节点为扩展结点时即为问题的最优值，算法结束。

6.3 0-1背包问题

b=背包已有物品价值；**cleft**=背包剩余容量，从第**i**~**n**物品为剩余的物品，用剩余物品装满剩余背包容量的背包的贪心算法（主要代码如下）

```
while (i <= n && w[i] <= cleft) // n表示物品总数，cleft为剩余空间
```

```
{
```

```
    cleft -= w[i];
```

```
    b += p[i];
```

```
    i++;
```

```
}
```

```
if (i <= n) b += p[i] / w[i] * cleft; // 装填剩余容量装满背包
```

```
return b; //b为上界函数值
```

上界函数bound (int i) 的计算

贪心算法中的背包问题

6.3 0-1背包问题

- **优先队列中的结点信息包含：**

当前背包价值、重量；

结点的优先级（价值上界）；

左孩子标志；

父结点地址；

本结点对应的下一个要装入背包的物品编号；

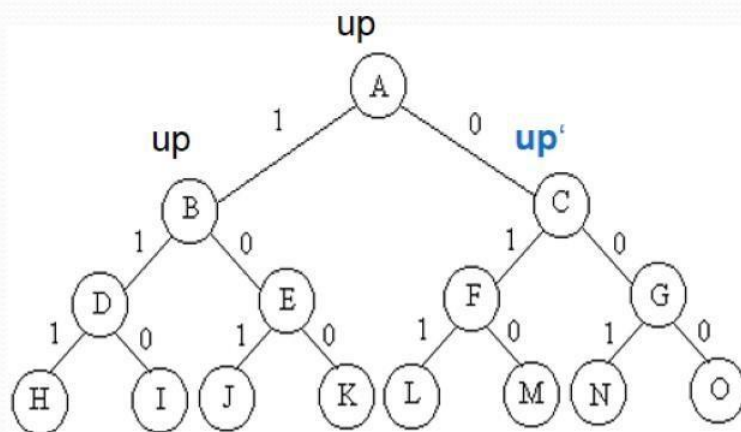
6.3 0-1背包问题

搜索算法的主代码

```
while (i != n+1) { // 非叶结点
    // 检查当前扩展结点的左儿子结点
    Typew wt = cw + w[i];
    if (wt <= c) { // 左儿子结点为可行结点
        if (cp+p[i] > bestp) bestp = cp+p[i];
        AddLiveNode(up, cp+p[i], cw+w[i], true, i+1);
        up = Bound(i+1);
    }
    // 检查当前扩展结点的右儿子结点
    if (up >= bestp) // 右子树可能含最优解
        AddLiveNode(up, cp, cw, false, i+1);

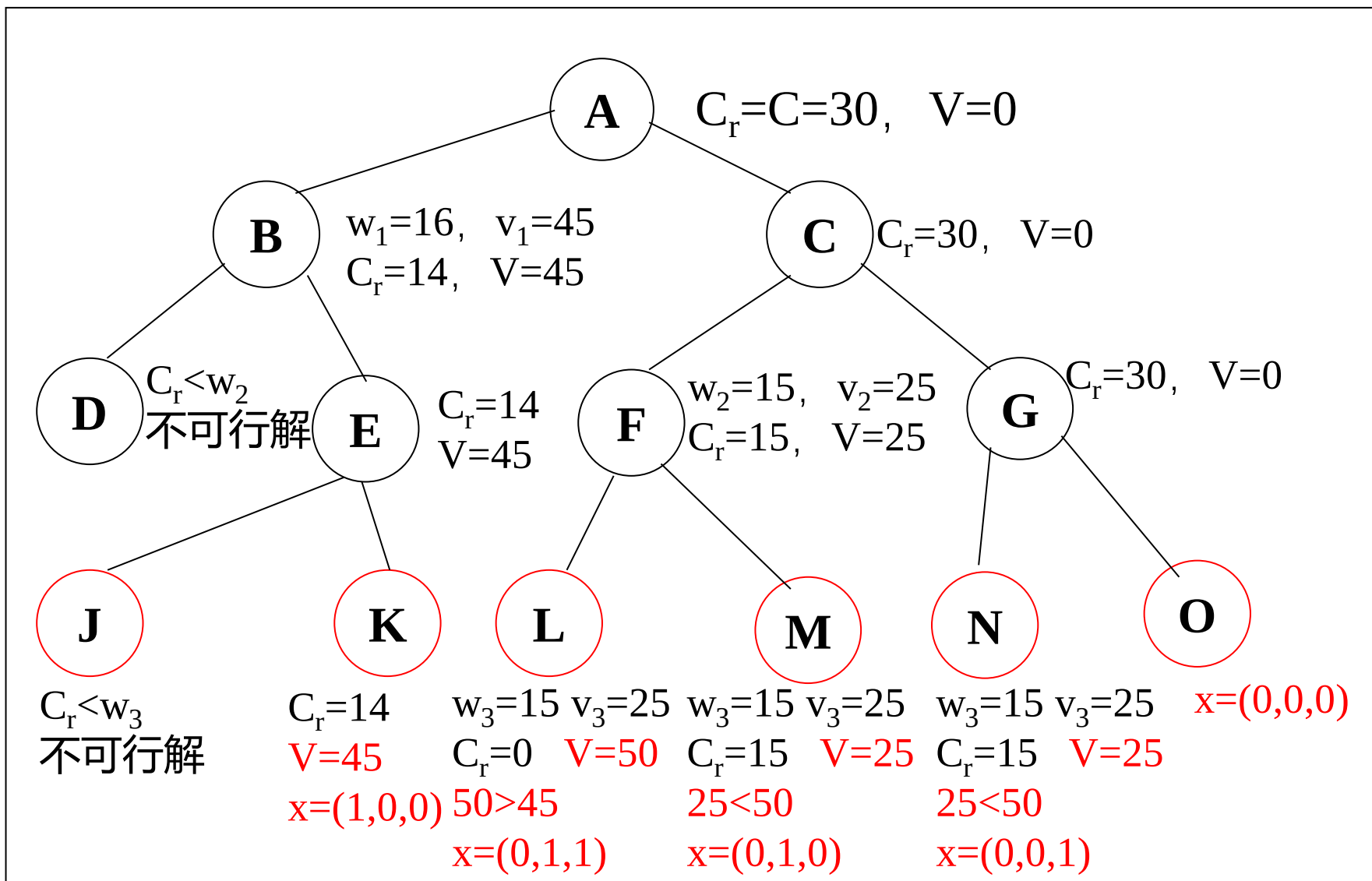
    // 取下一个扩展节点（略）；
}
```

注意优先级up的变化规律



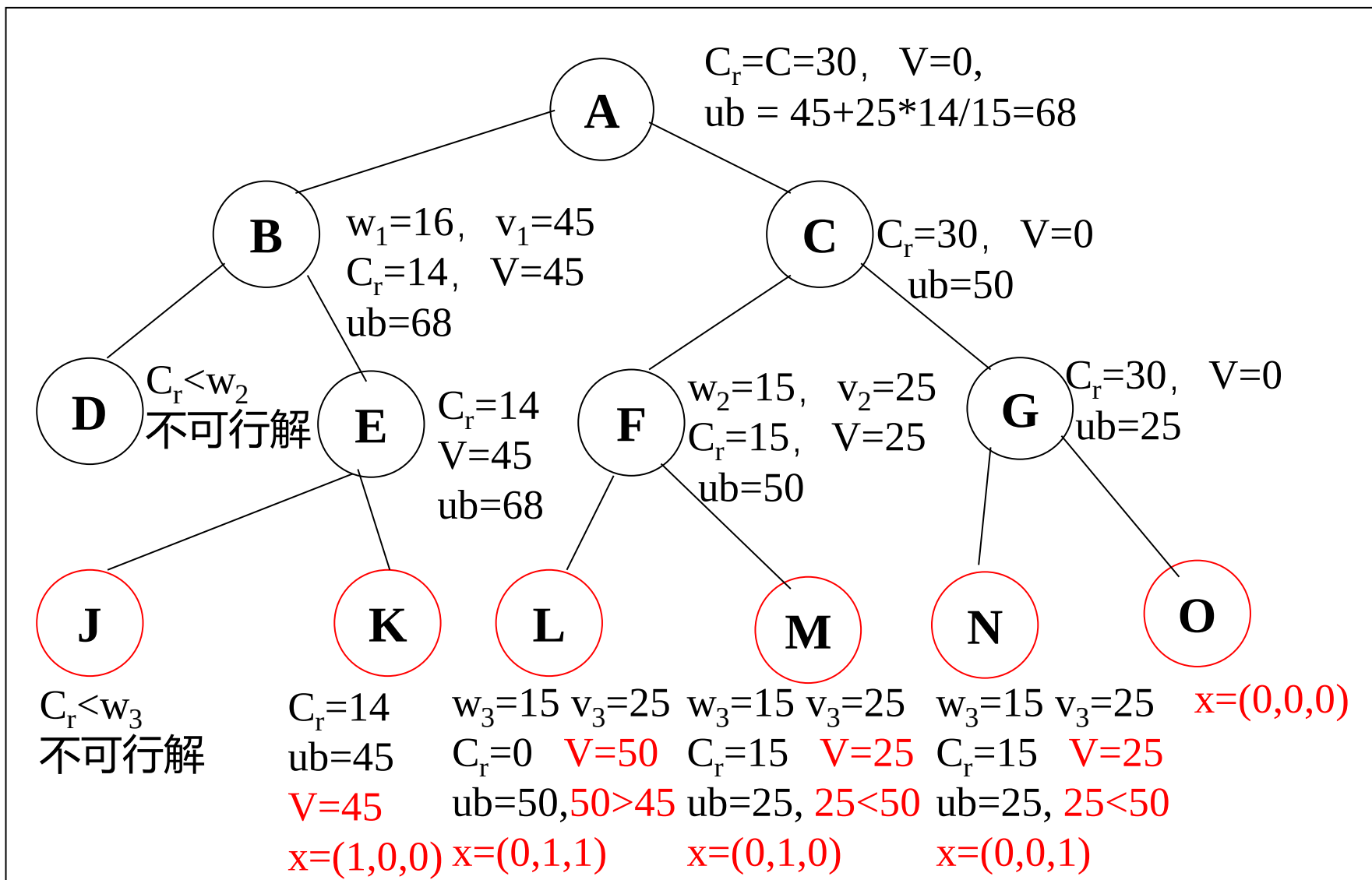
示例1 0-1背包问题（队列式分支限界）

$n=3, C=30, w=\{16, 15, 15\}, v=\{45, 25, 25\}$



示例1 0-1背包问题（最大优先队列式分支限界）

$n=3, C=30, w=\{16, 15, 15\}, v=\{45, 25, 25\}$



6.3 0-1背包问题

思考题：

如果优先级设定为当前为止的背包内物品价值，算法是否可以在一个叶子结点成为一个扩展结点时算法结束？

广度优先是（ ）的一搜索方式。

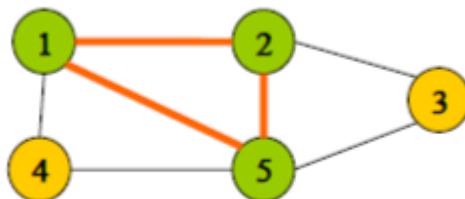
- ☒ A 分支界限法
- ☐ B 动态规划法
- ☐ C 贪心法
- ☐ D 回溯法

提交

6.4 最大团问题

1.问题描述

- 给定无向图 $G=(V, E)$ 。如果 $U \subseteq V$ ，且对任意 $u, v \in U$ 有 $(u, v) \in E$ ，则称 U 是 G 的完全子图。 G 的完全子图 U 是 G 的团当且仅当 U 不包含在 G 的更大的完全子图中。 G 的最大团是指 G 中所含顶点数最多的团。
- 子集 $\{1, 2\}$ 是 G 的大小为2的完全子图。但不是团，因为它被 G 的更大的完全子图 $\{1, 2, 5\}$ 包含。 $\{1, 2, 5\}$ 、 $\{1, 4, 5\}$ 、 $\{2, 3, 5\}$ 是 G 的最大团。

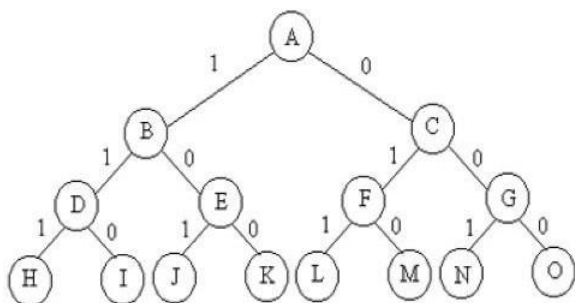


6.4 最大团问题

2. 上界函数

- 用变量cliqueSize表示与该结点相应的团的顶点数；level表示结点在子集空间树中所处的层次；用cliqueSize + 剩余定点数作为顶点数上界upperSize的值 它是优先队列中元素的优先级。

- 解空间树：

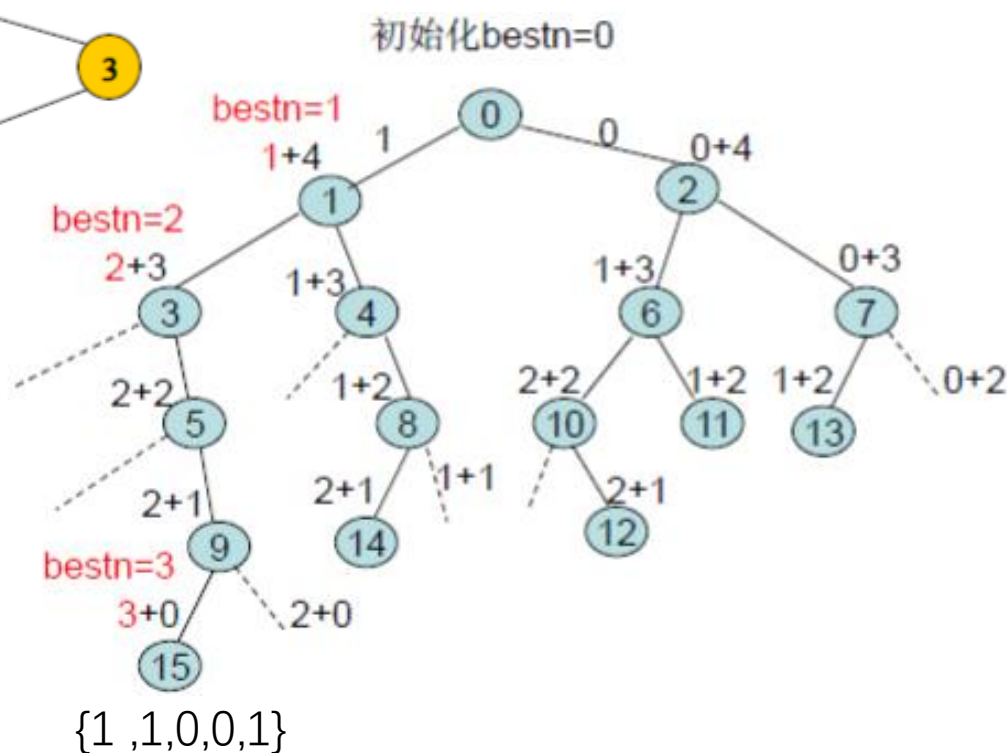


6.4 最大团问题

3. 算法思想

- (1) 子集树的根结点是初始扩展结点，其cliqueSize的值为0。
- (2) 在扩展内部结点时，首先考察其左儿子结点。在左儿子结点处，将顶点i加入到当前团中，并检查该顶点与当前团中其他顶点之间是否都有边相连。有则将它插入活结点优先队列，否则就不是可行结点。
- (3) 接着继续考察当前扩展结点的右儿子结点。当upperSize>bestn时，右子树中可能含有最优解，此时将右儿子结点插入到活结点优先队列中。
- (4) 从优先队列中取下一个活结点，重复(2)(3)，直到一个叶子结点成为扩展结点，找到最优解，算法结束

6.4 最大团问题



活结点优先队列

0				
1	2			
3	2	4		
2	4	5		
4	5	6	7	
5	6	7	8	
6	7	8	9	
10	7	8	9	11
7	8	9	11	12
8	9	11	12	13
9	11	12	13	14
11	12	13	14	15
12	13	14	15	
13	14	15		
14	15			
15				

分支限界是广度优先，一层一层往下找，中间不断排除不符合的，找到最下面一层就得到了最优解。

6.5 旅行售货员问题

1. 问题描述

- 某售货员要到若干城市去推销商品，已知各城市之间的路程(或旅费)。他要选定一条从驻地出发，经过每个城市一次，最后回到驻地的路线，使总的路程(或总旅费)最小。

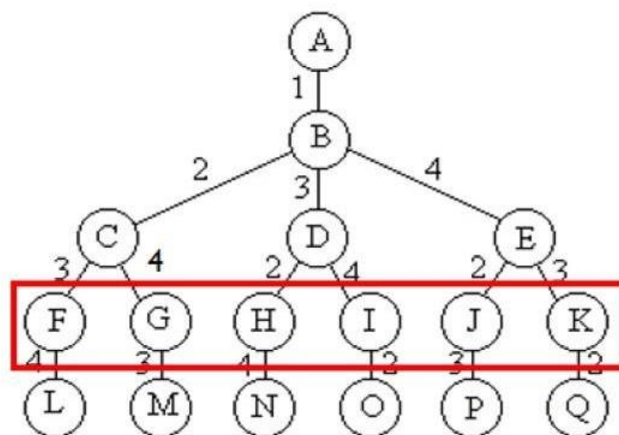
6.5 旅行售货员问题

2.解空间树是排列树

- (1) 指定一个城市作为出发城市
- (2) 第 $n-1$ 层结点看做叶子结点

n=4 TSP问题

X[1]
X[2]
X[3]
X[4]



6.5 旅行售货员问题

3. 算法描述

- 算法开始时创建一个最小堆，用于表示活结点优先队列。
- 堆中每个活结点的优先级为： $cc + lcost$ 。 cc 为出发城市到当前城市的路程（或费用）， $lcost$ 是当前顶点(当前城市)最小出边+剩余顶点（城市）最小出边和（禁忌除外）。
- 每次从优先队列中取出一个活结点成为扩展结点（ s 层结点），
- 当 $s = n - 2$ 时，扩展出的结点是排列树中某个叶子结点的父结点。如该叶结点相应一条可行回路且费用小于当前最优解 $bestc$ ，则将该结点插入到优先队列中，否则舍去该结点

- **当 $s < n-2$ 时**，产生当前扩展结点的所有儿子结点。计算可行儿子结点的优先级 $cc+lcost$ 及相关信息。当 $cc+lcost < bestc$ 时，将这个可行儿子结点插入到活结点优先队列中。
- 该扩展过程一直持续到优先队列中取出的活结点是一个叶子结点为止。
- **最小出边（禁忌除外）的解释**

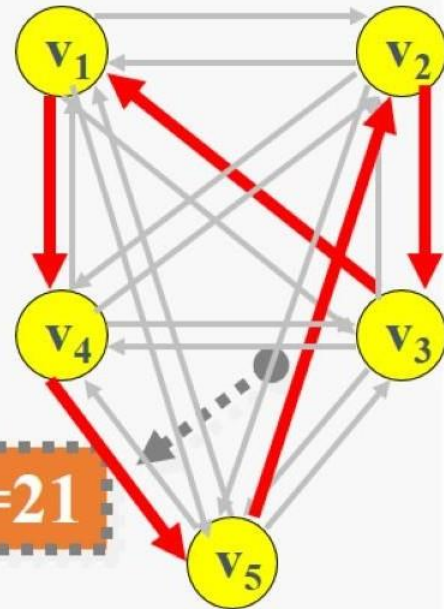
对于刚扩展出的顶点，其前已选的所有顶点是禁忌的（不能选）。

对于未扩展出的顶点，其前已选的（**顶点1除外**）的顶点是禁忌的

算法演示

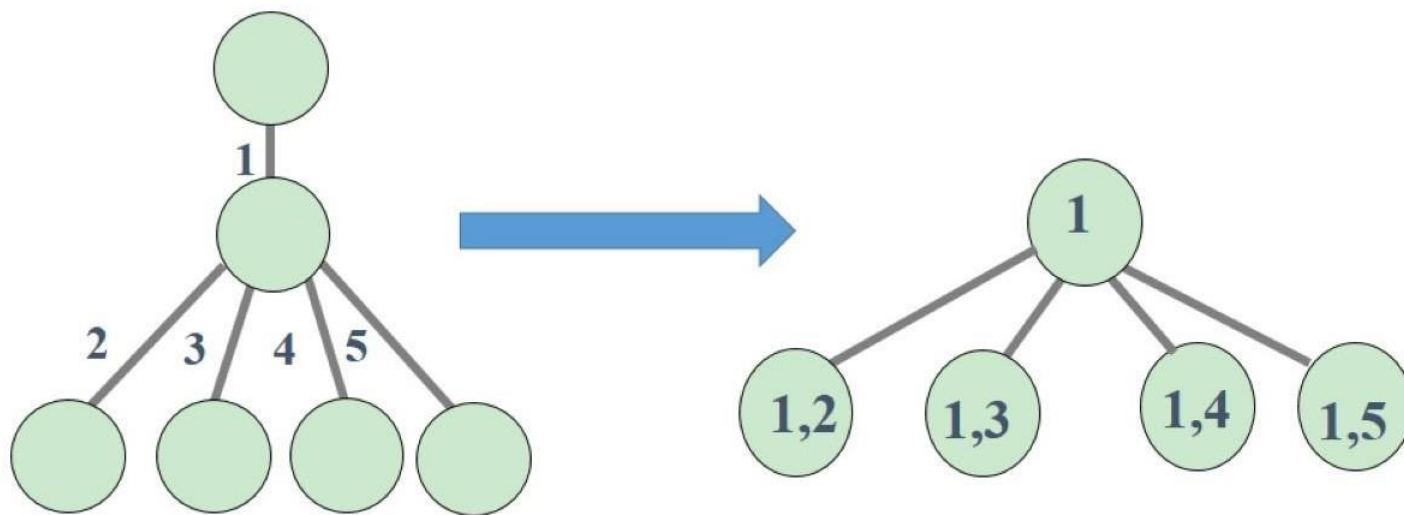
0	14	4	10	20
14	0	7	8	7
4	5	0	7	16
11	7	9	0	2
18	7	17	4	0

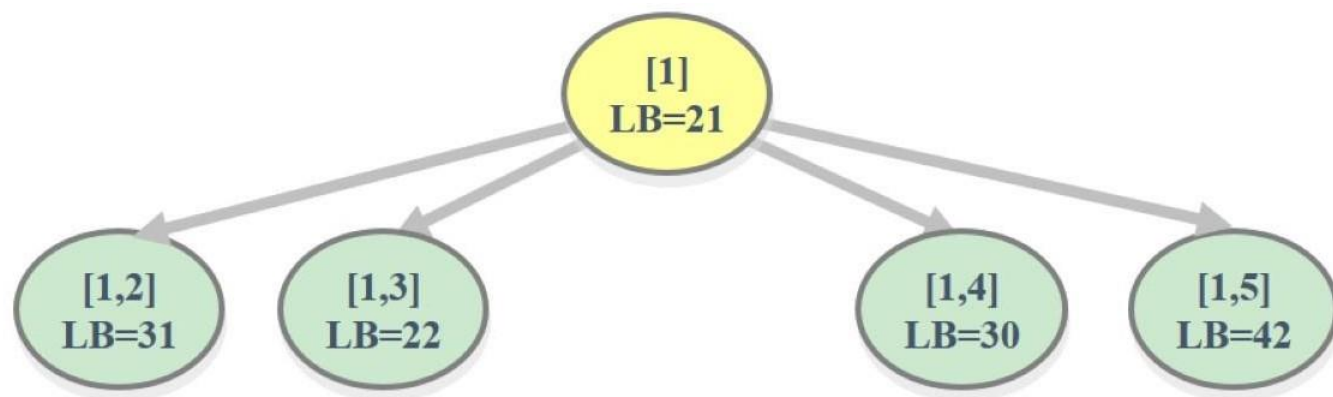
最小出边和之成本下界LB=21



LB=当前路径长度+当前扩展出的顶点及其后顶点可选最小出边和。

为动画演示状态空间改变为如图所示

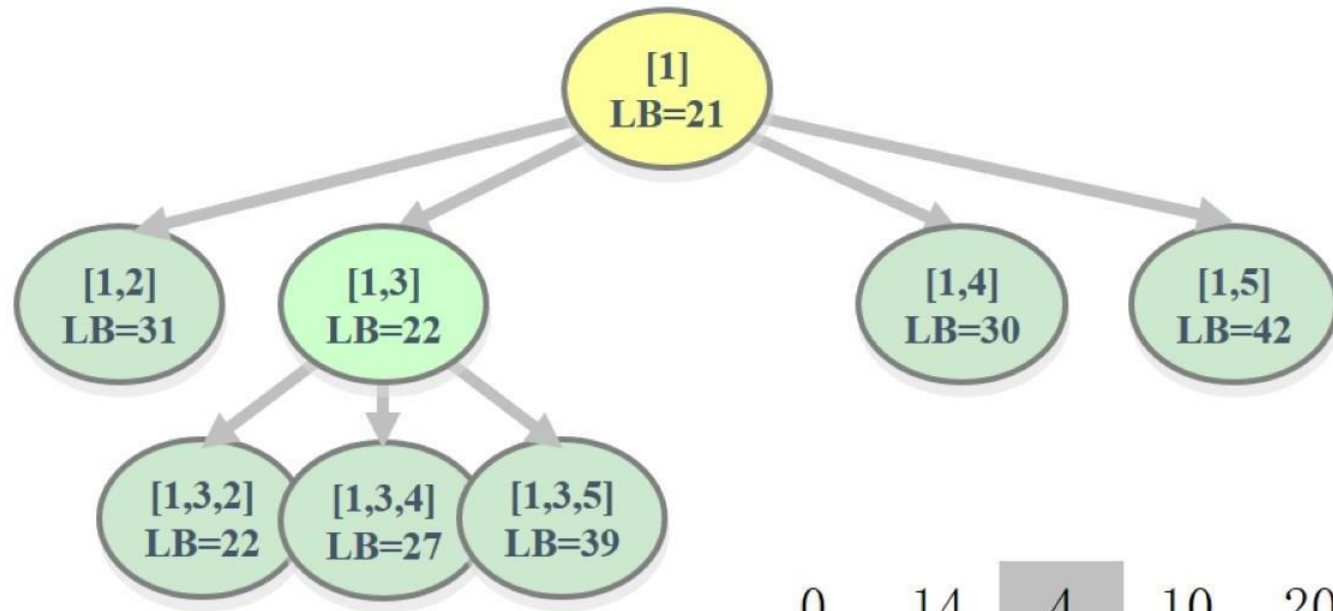




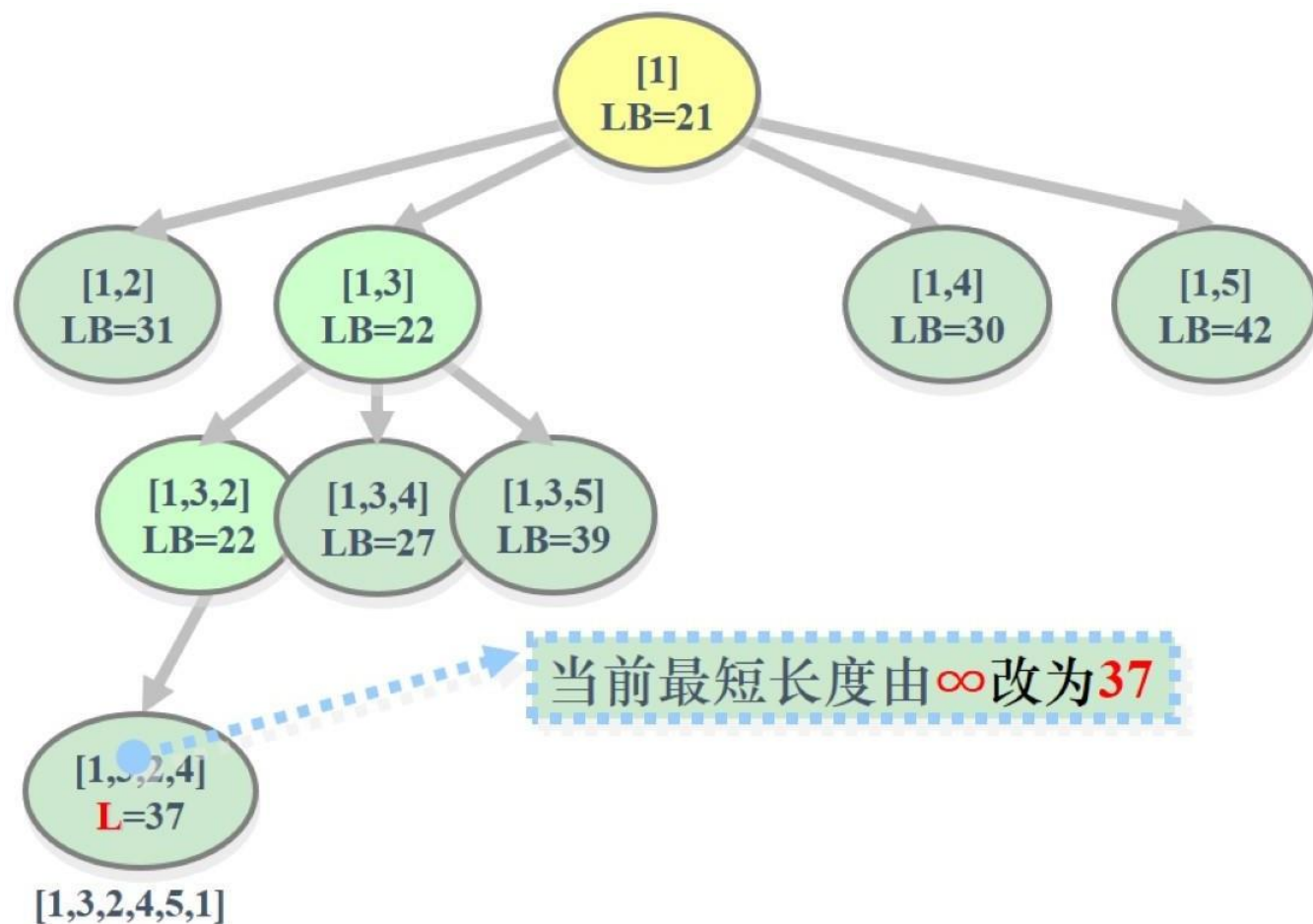
第一个孩子结点的优先级

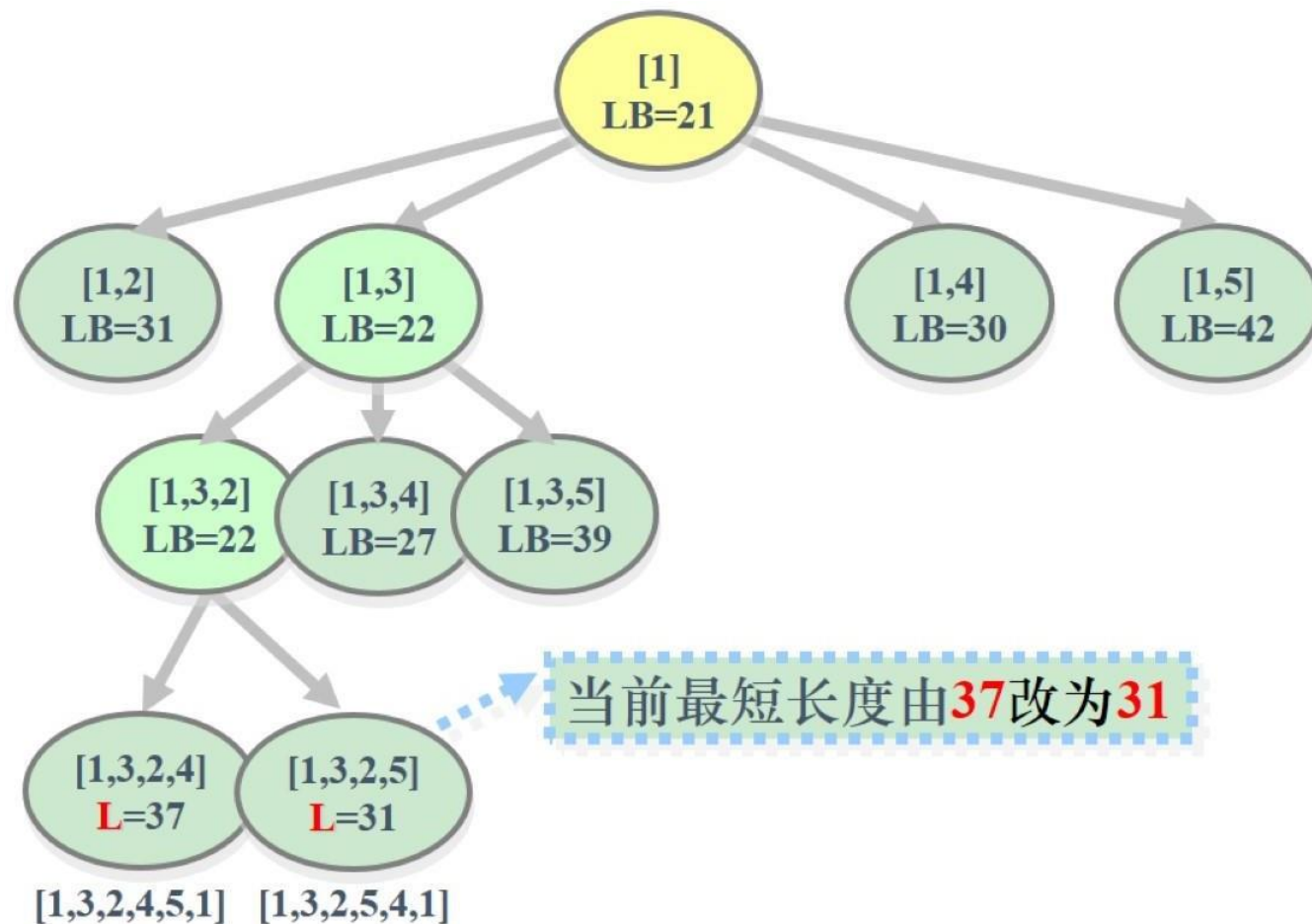
$LB = cc + lcost = 14 + 7(2\text{号城市最小出边}) + 4(3\text{号城市最小出边}) + 2(4\text{号城市最小出边}) + 4(5\text{号城市最小出边}) = 31$

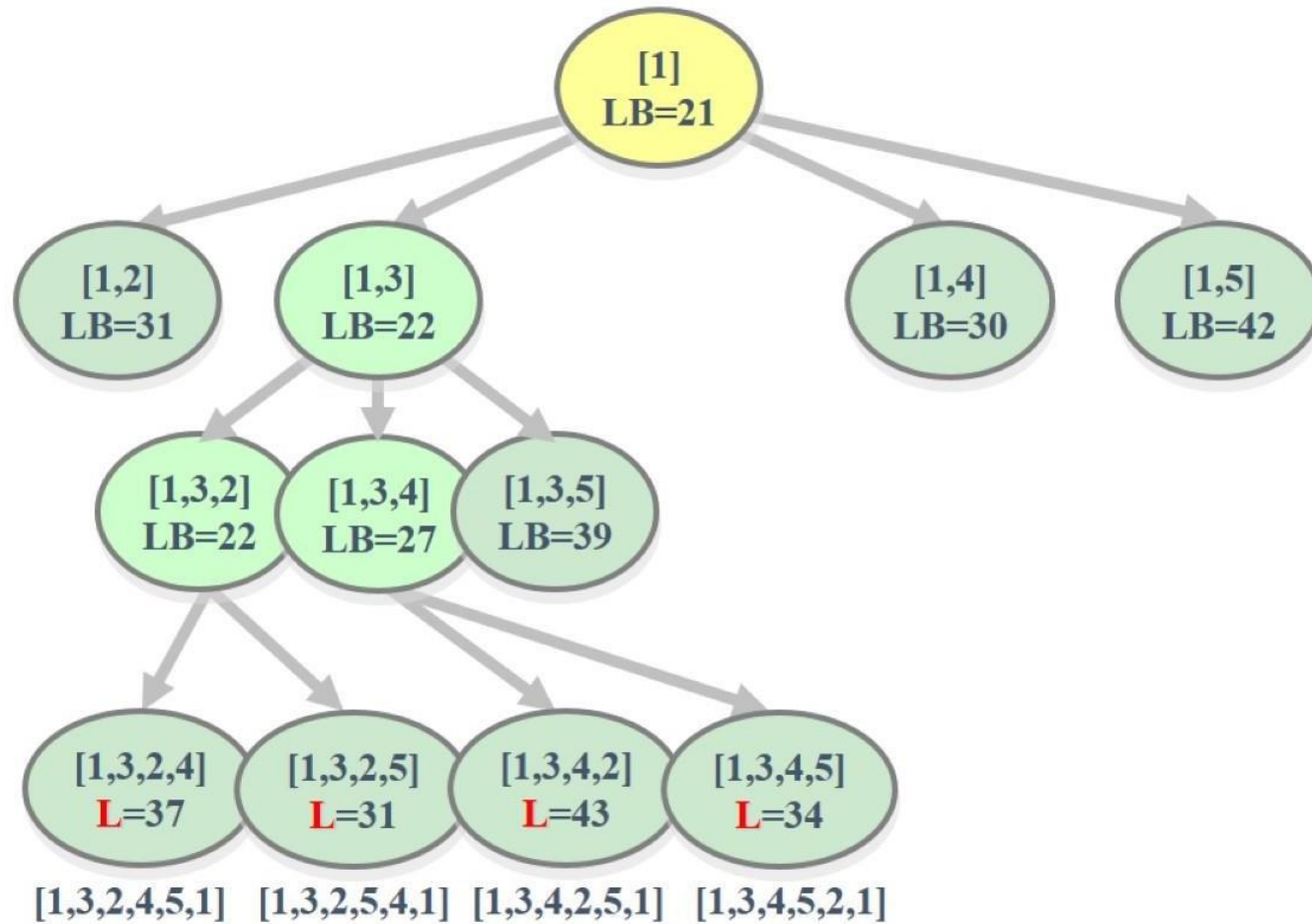
0	14	4	10	20
14	0	7	8	7
4	5	0	7	16
11	7	9	0	2
18	7	17	4	0

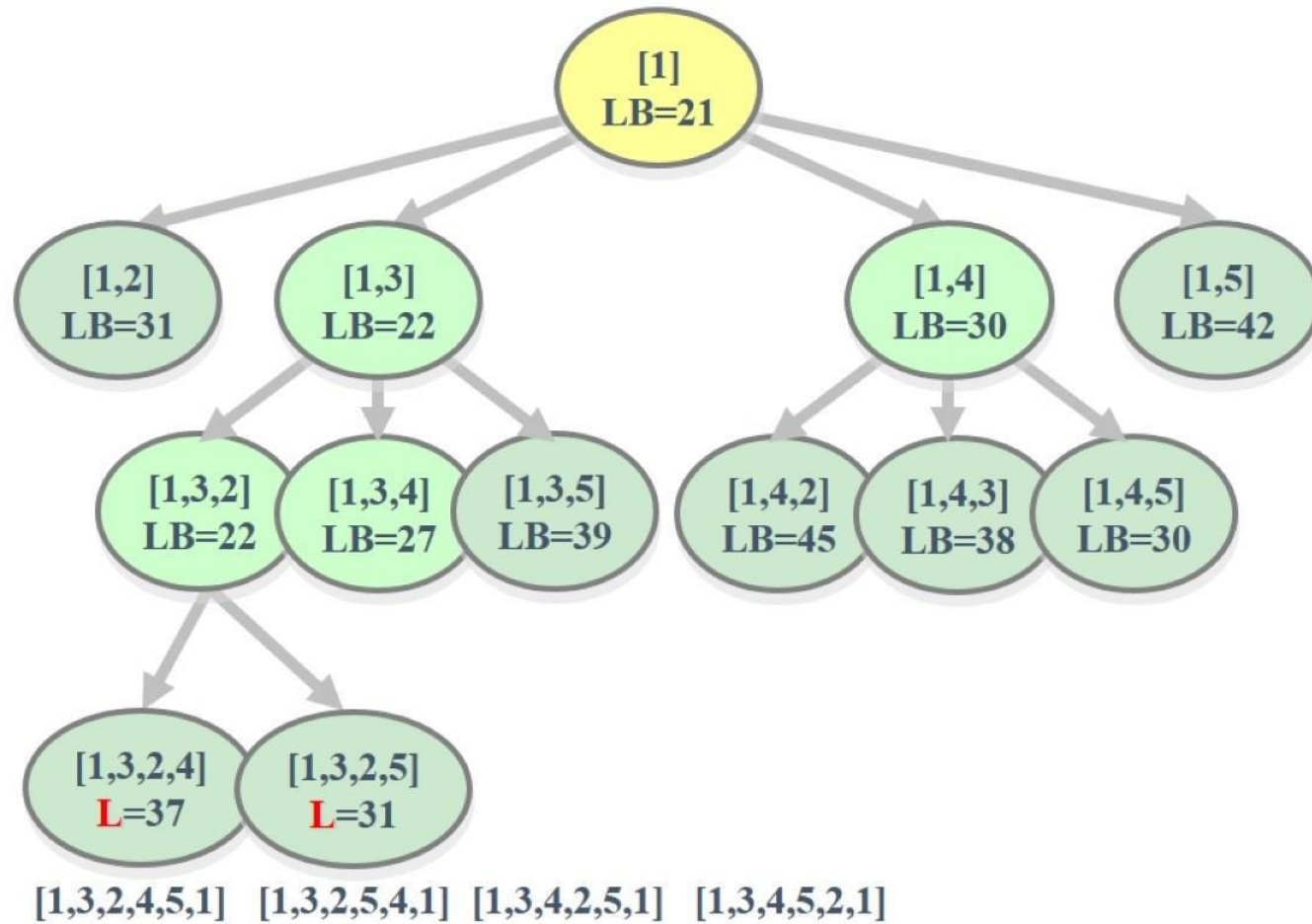


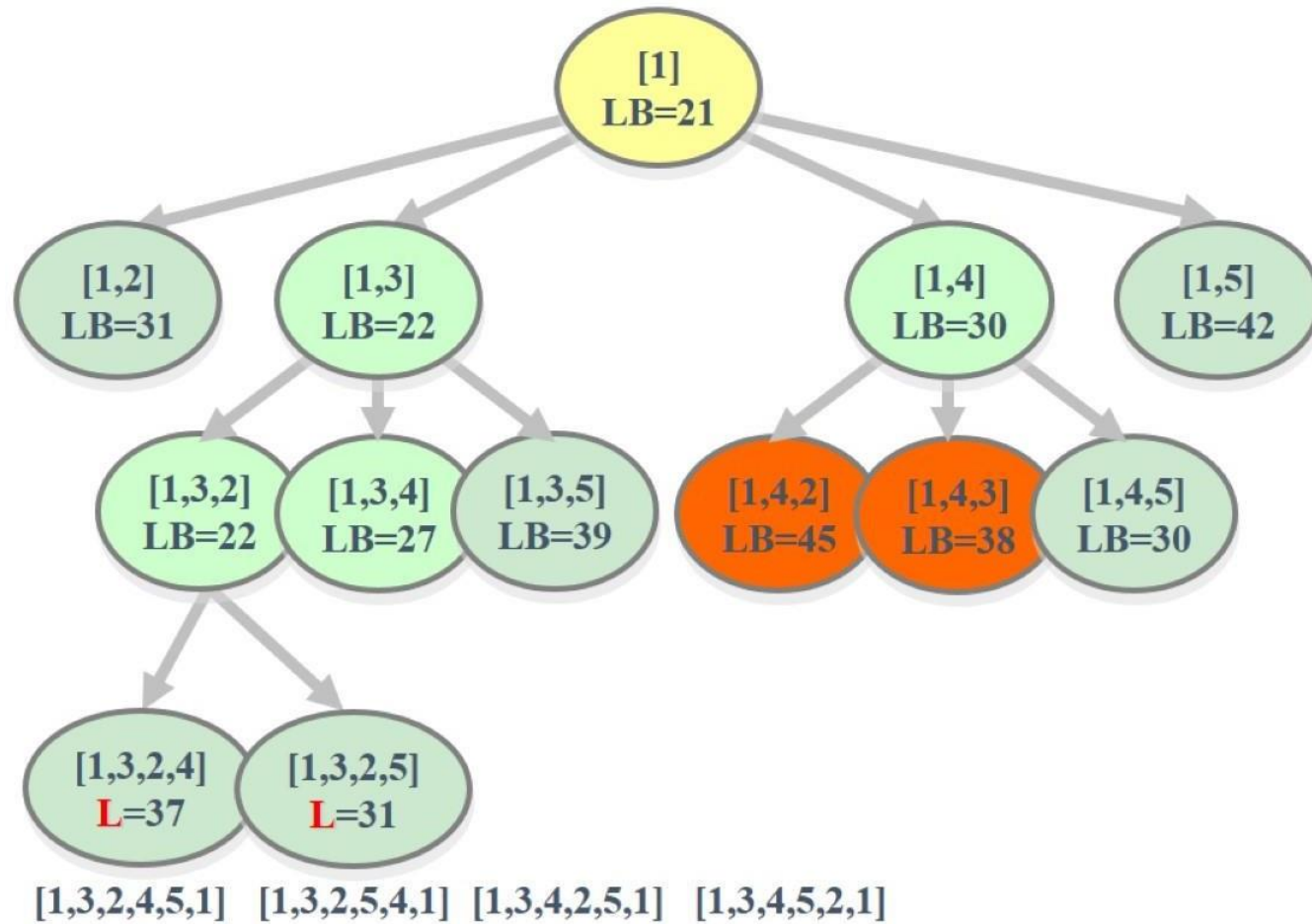
0	14	4	10	20
14	0	7	8	7
4	5	0	7	16
11	7	9	0	2
18	7	17	4	0

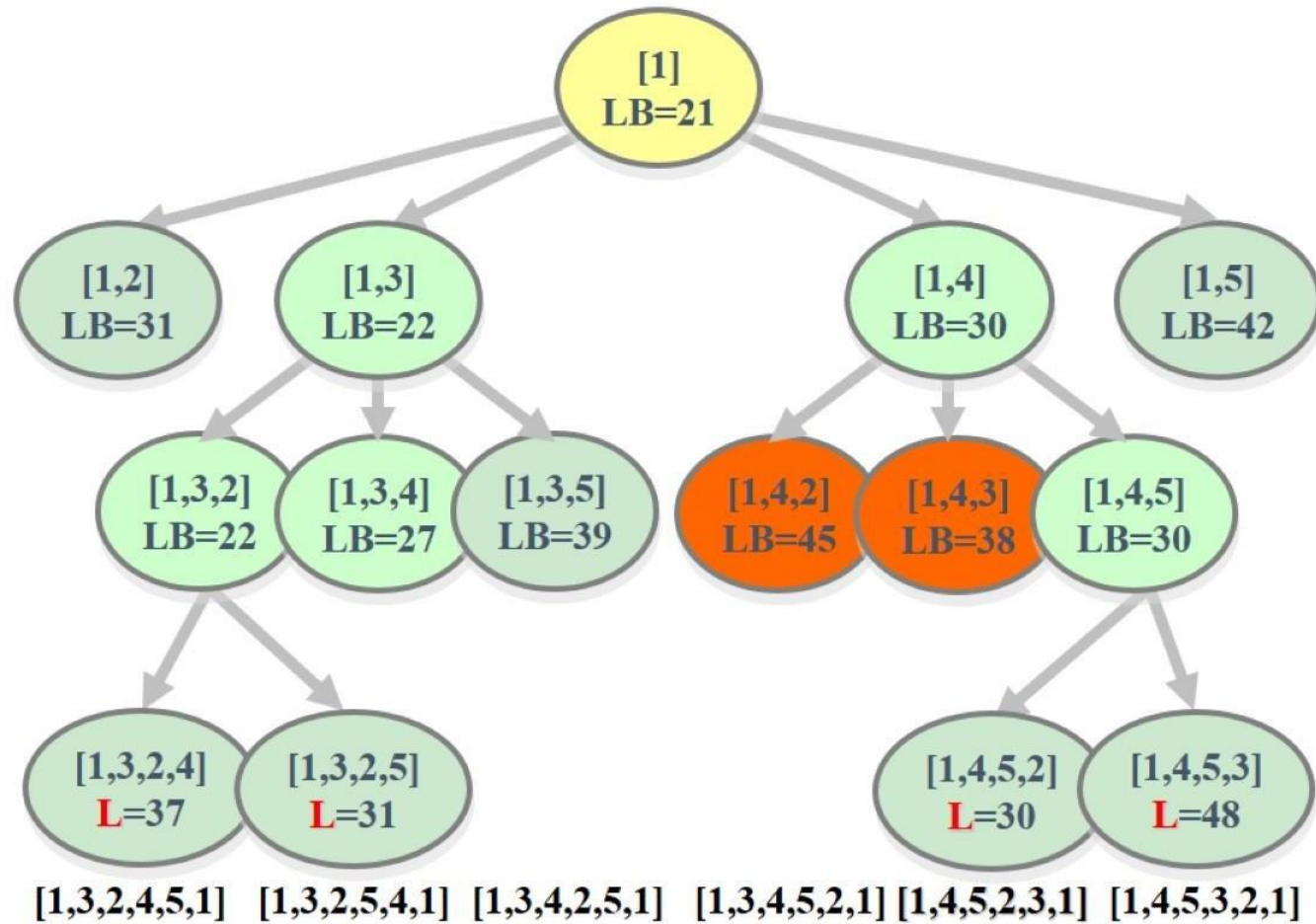


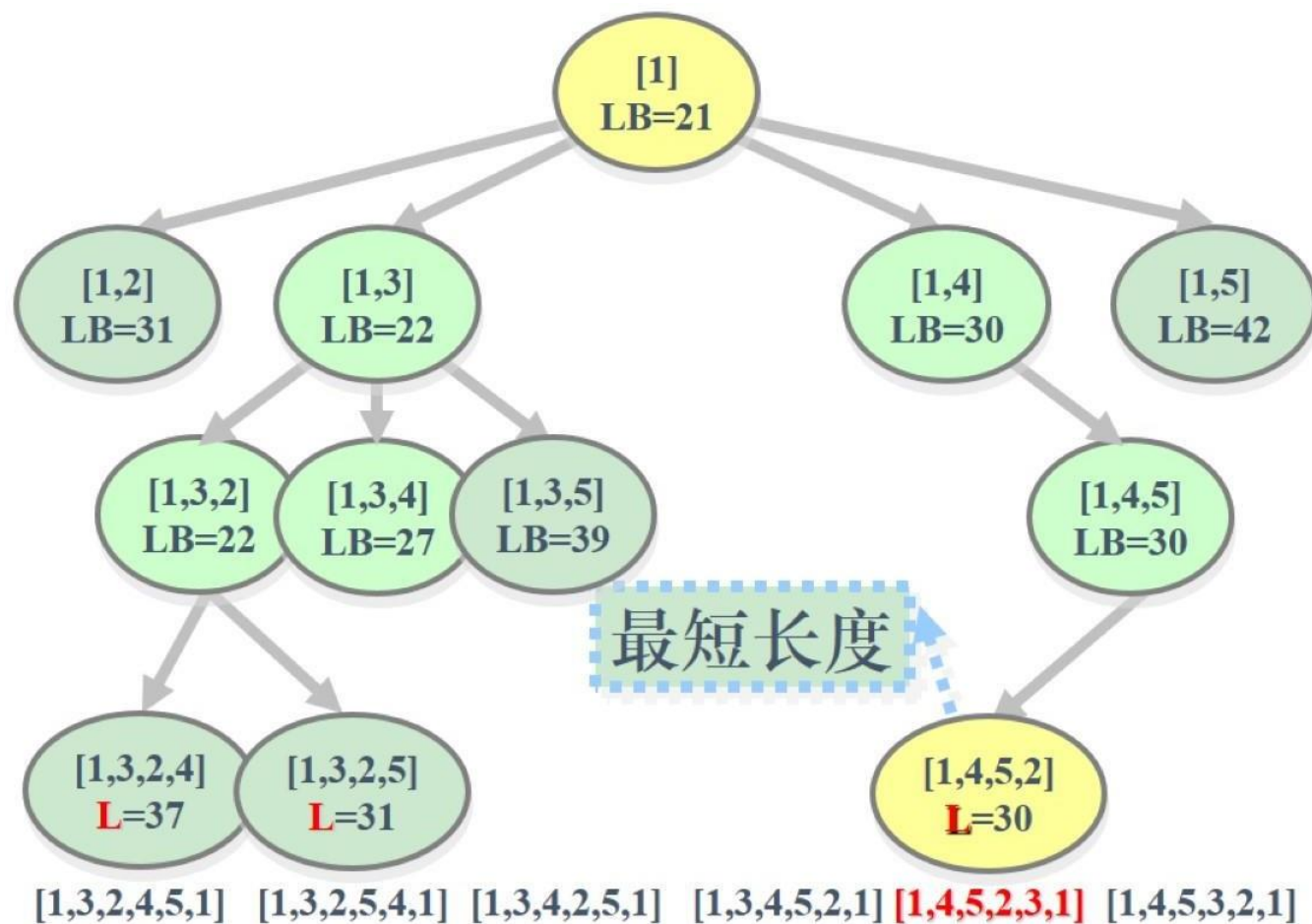












Algorithm 1: 分支限界算法实现旅行商问题

Input: 图的邻接矩阵: G

Output: 最优解和最优值

初始化最小堆;

while 遍历每一层 $E.s < (n-1)$ **do**

if 当前扩展结点是叶结点的父结点 (到了 $n-2$ 层) **then**

if 形成回路且当前值优于最优值或还未产生最优值 **then**

 更新最优解;

 更新最优值;

else

 舍弃需要进一步搜索的节点;

else

for 遍历这个节点的所有扩展节点 **do**

if 这个点可以进入下一个未访问的点 **then**

 更新当前费用;

 更新最小出边费用和;

 更新下界 (限界函数);

if 子树可能含最优解 **then**

 节点插入最小堆;

 完成当前节点的扩展, 清除该节点;

if 最小堆已经空 **then**

 结束循环

 从最小堆中弹出队首节点 E ;

```

#define NoEdge -1
#define NN 50 // 可执行的最大的顶点个数
using namespace std;
int n;          // 图的顶点个数
int adjMatrix[NN][NN]; // 图的邻接矩阵
int v[50];      // 最优解
int bestC;      // 最优值
int num = 0;    // 节点序号

/*****
* 函数描述： 数据输入以及内存的初始化
*****/
void input()
{
    cin >> n; // 输入顶点个数
    int k;
    // 邻接矩阵的内存初始化
    memset(adjMatrix, NoEdge, sizeof(adjMatrix));
    cin >> k; // 输入边的个数;
    int p, q, len;
    // 初始化邻接矩阵
    for (int i = 1; i <= k; ++i)
    {
        cin >> p >> q >> len;
        adjMatrix[p][q] = len;
        adjMatrix[q][p] = len;
    }
}

```

数据结构及图输入

```

/*****
* 类描述： 最小堆（队列中元素类型）
* 参数描述：
x, 用于记录当前解;
s, 表示节点在排列树中的层次，从排列树的根节点到该
节点的路径为x[0:s],
需要进一步搜索的顶点是x[s+1:n-1]。
cc, 表示当前费用,
rcost, 是子树费用的下界,
lcost, 是x[x:n-1]中顶点最小出边费用和。
*****/
class MinHeapNode
{
public:
    char name; // 节点的序号
    int lcost, // x[s:n-1]中顶点最小出边费用和
        rcost, // 子树费用的下界
        cc; // 当前费用
    int s, // 根节点到当前节点的路径为x[0:s]
        *x; // 需要进一步搜索的顶点是x[s+1:n-1]

    // 构造节点并递增序号
    MinHeapNode()
    {
        num += 1;
        name = num + 'A';
    }
    // 最小堆中使用下界排序
    bool operator<(const MinHeapNode &MH) const
    { return rcost > MH.rcost; }
}

```

```
/******
```

* 算法描述：核心算法

算法开始时创建一个最小堆，表示活节点优先队列。堆中每个节点的lcost值是优先队列的优先级。接着计算出图中每个顶点的最小费用出边并用Minout记录。如果所给的有向图中某个顶点没有出边，则该图不可能有回路，算法即告结束。如果每个顶点都有出边，则根据计算出的Minout作算法初始化。

```
*****/
```

//搜索排列空间树

```
while (E.s < n - 1) //非叶节点
```

```
{
```

```
    if (E.s == n - 2) // 当前扩展节点是叶节点的父节点，判断构成的回路是否最优
```

```
    {
```

```
        if (adjMatrix[E.x[n - 2]][E.x[n - 1]] != NoEdge && adjMatrix[E.x[n - 1]][1] != NoEdge &&
```

```
            (E.cc + adjMatrix[E.x[n - 2]][E.x[n - 1]] + adjMatrix[E.x[n - 1]][1] < bestC || bestC == NoEdge))
```

```
        { // 如果更优，则更新费用更小的路
```

```
            cout << "\n||||| 到达叶子节点的父节点 —— 并更新最优解 |||||" << endl;
```

```
            E.printNode(pq);
```

```
            bestC = E.cc + adjMatrix[E.x[n - 2]][E.x[n - 1]] + adjMatrix[E.x[n - 1]][1];
```

```
            E.cc = bestC;
```

```
            E.rcost = bestC;
```

```
            E.s++;
```

```
            pq.push(E);
```

```
        }
```

```
    else
```

```
    {
```

```
        cout << "\n||||| 到达叶子节点的父节点 —— 不更新 |||||" << endl;
```

```
        E.printNode(pq);
```

```
        delete[] E.x; // 舍弃需要进一步搜索的节点
```

```
    }
```

```
}
```

```

else // 产生当前扩展节点儿子节点
{
    cout << "\n***** 开始一个新节点扩展 *****\n" << endl;
    for (i = E.s + 1; i < n; i++) // 广度优先搜索, 进行子节点的扩展
    {
        MinHeapNode N;
        if (adjMatrix[E.x[E.s]][E.x[i]] != NoEdge) // E.x[E.s] 是当前要扩展的父节点, E.x[i] 是被遍历的子节点
        {
            // 可行儿子节点
            cc = E.cc + adjMatrix[E.x[E.s]][E.x[i]]; // 当前费用 = 之前费用 + 新增费用
            lcost = E.lcost - MinOut[E.x[E.s]]; // 更新最小出边费用和
            lb = cc + lcost; // 下界 (限界函数)
            if (lb < bestC || bestC == NoEdge) // 子树可能含最优解 节点插入最小堆
            {
                N.s = E.s + 1; // 进入下一层
                N.cc = cc;
                N.rcost = lb;
                N.lcost = lcost;
                N.x = new int[n];
                for (j = 0; j < n; j++)
                    N.x[j] = E.x[j];
                N.x[E.s + 1] = E.x[i]; // 获得新的路径【换位】
                N.x[i] = E.x[E.s + 1];
                pq.push(N); // 加入优先队列
                N.printNode(pq);
            }
        }
    }
    delete[] E.x; // 完成节点扩展
}
if (pq.empty()) // 堆已空
    break;
E = pq.top(); // 取下一扩展节点
pq.pop();
}

```

实验6-2: 旅行商问题(分支限界法)

- 某售货员要到 n 个城市去推销商品, 已知各城市之间的路程, 请问他应该如何选定一条从城市1出发, 经过每个城市一遍, 最后回到城市1的路线, 使得总的周游路程最小。
- 示例: 结点数 4 边数 6
输入:
4
6
1 2 30
1 3 6
1 4 4
2 3 5
2 4 10
3 4 20
- 要求:
1 填空完成程序;
2 输出最少旅行费用。
*3 报告中指出程序使用是队列还是优先队列。

优先队列式分支限界法求解旅行售货员问题时，活结点表的组织形式是（ ）。

- ☒ A 最小堆
- ☐ B 最大堆
- ☐ C 栈
- ☐ D 数组

提交

6.6 小结

1. 队列式：活结点表是一个队列，新扩展出的满足条件的活结点追加在队尾。这里需要加入层次标志（如-1），或记录下层编号在活结点中。
2. 优先队列式：活结点表是优先队列，一般使用堆（大顶堆或小顶堆）存储，优点是只需要 $O(\log n)$ 时间复杂性完成插入或者删除（取堆顶结点，即优先级最高的结点）
3. 构造解方法：活结点通过记录其父节点地址，及左孩子标志去构造最优解。在找到最优值时，回溯方法找到最优解。**也可在扩展出的结点中记录构造的解，如问题规模较大时，应考虑压缩存储。**

6.6 小结

4.分支限界法的剪枝方法：

- (1) 对于子集树，左右分支剪枝策略不同，
- (2) 对于排列树， n 叉树，剪枝策略是相同的。

5. 算法的结束控制：

- (1) 队列式分支限界，活结点表为空
- (2) 优先队列式，叶子结点成为扩展结点(在确认后面的活结点不存在更好的解)或队列为空。通常叶子结点加入优先队列中。